Statistical and Computational Models

Lecture 20: Dimension reduction

Daniel de Kadt

Department of Government, Cornell University · Fall 2026

Roadmap

Learning without an outcome

One component

Finding the loadings

Many components

A simulated legislature

Suppose we are studying a legislature with 150 legislators.

Having read a few books about the setting, we know it's important to measure each legislator's political party and their region:

Legislator	Party	Region	RC1	RC2	RC3		RC300
21	Left	South	1	0	0	...	1
1	Left	North	1	0	0	...	0
66	Center	North	0	1	1	...	0
91	Right	North	0	1	1	...	0

After a few trips to the archives we are also able to measure each legislator's vote (yea = 1, nay = 0) on 300 different roll calls.

Write y_{ij} for legislator i's vote on roll call j. What can we learn about the legislature using our data?

Three questions

As always, we should start by writing down research questions:

1. What cleavages exist? Is there a left–right conflict, or are there other **dimensions** of disagreement?
2. Who votes with whom? Are parties voting blocs, or are there deeper **clusters**?
3. Where does each legislator stand? Do they have **latent** positions we can measure?

Until now we have estimated $\mathbb{E}[Y \mid \boldsymbol{x}]$: one variable was the outcome, and the rest were features. Here no single vote is the outcome of interest; instead, we want to learn about all 300 together.

This is **unsupervised learning**.

Roadmap

Learning without an outcome

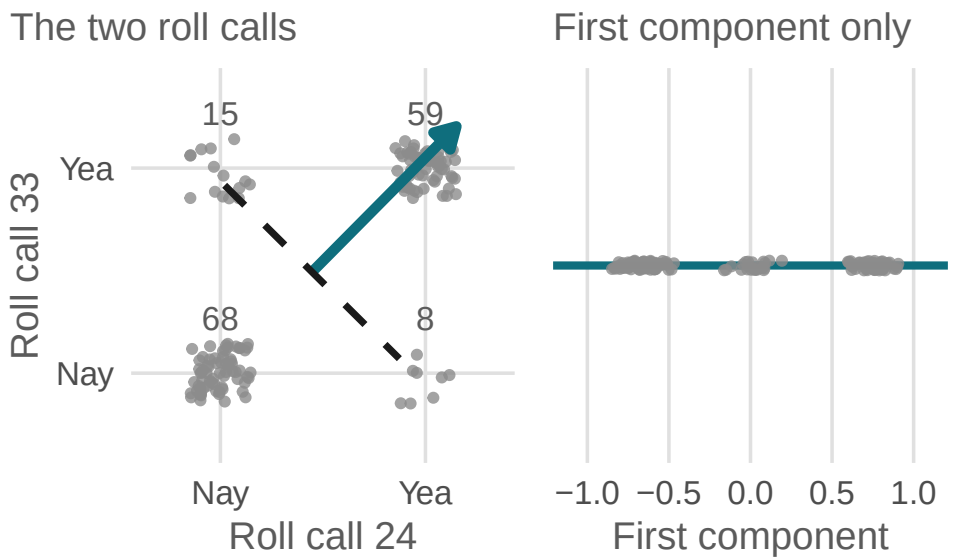
One component

Finding the loadings

Many components

Two roll calls

Start with the first question: how many dimensions of disagreement are there? If there were only one, a single number per legislator could summarize all their votes. Let's begin by looking at just two roll calls:



Now flatten onto the arrow's direction, so each legislator gets just one number – their **score**.

Scores from two roll calls

Let's see how this works mathematically, for just these two roll calls (roll call 24 is $j = 1$, roll call 33 is $j = 2$).

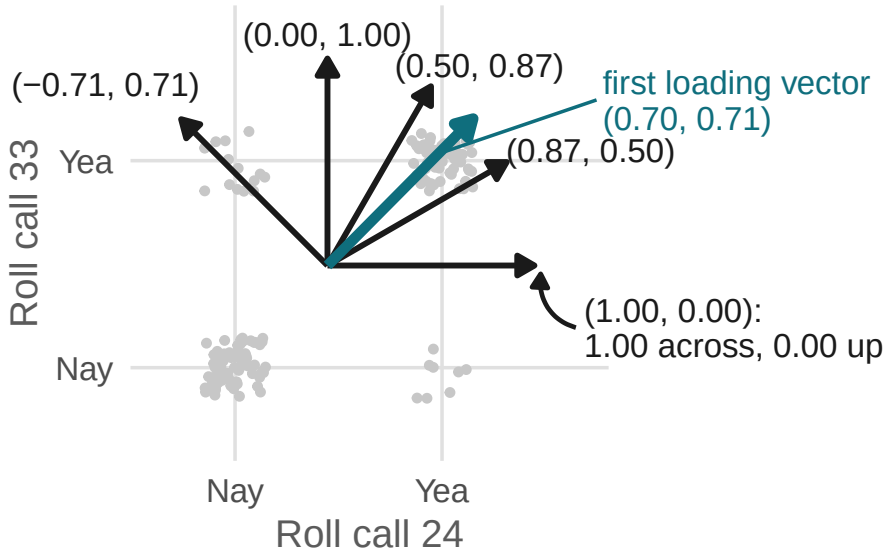
Each legislator's score is a weighted sum of their two votes, each measured from its mean ($\bar{y}_j$, the share voting yea):

$$z_{i1} = \underbrace{\phi_{11}}_{\text{weight on roll call 1}} (y_{i1} - \bar{y}_1) + \phi_{21} \underbrace{(y_{i2} - \bar{y}_2)}_{\text{how unusual } i \text{ is on roll call 2}}$$

What do ϕ_{11} and ϕ_{21} represent? These weights, called **loadings**, set the importance of each roll call, and in so doing set the direction of the first principal component: the **loading vector** $\phi_1 = (\phi_{11}, \phi_{21})$.

Loadings as a vector

Each pair of loadings is a vector, (ϕ_{11}, ϕ_{21}), that points in its own direction from the mean of the two roll calls:



Read: ϕ_{11} is how far the vector's tip runs along roll call 24, and ϕ_{21} how far along roll call 33.

Note that because $\phi_{11}^2 + \phi_{21}^2 = 1$, every vector has length 1.

From votes to scores

For now let's take that direction as given: $\phi_{11} = 0.70$ and $\phi_{21} = 0.71$. In a moment we will return to finding these values. Each score is then:

$$z_{i1} = 0.70\,(y_{i1} - 0.45) + 0.71\,(y_{i2} - 0.49)$$

Let's do the calculation for one legislator from each corner:

y_{i1}	y_{i2}	$y_{i1} - \bar{y}_1$	$y_{i2} - \bar{y}_2$	Score, z_{i1}
1	1	0.55	0.51	0.75
0	1	−0.45	0.51	0.05
1	0	0.55	−0.49	0.04
0	0	−0.45	−0.49	−0.66

The two who split their votes land near 0: their yea and nay roughly cancel.

Why measure each vote from its mean? PCA looks for spread, and spread is measured around the mean. Without centering, the first direction would mostly track how often each roll call passes, not who disagrees with whom.

Roadmap

Learning without an outcome

One component

Finding the loadings

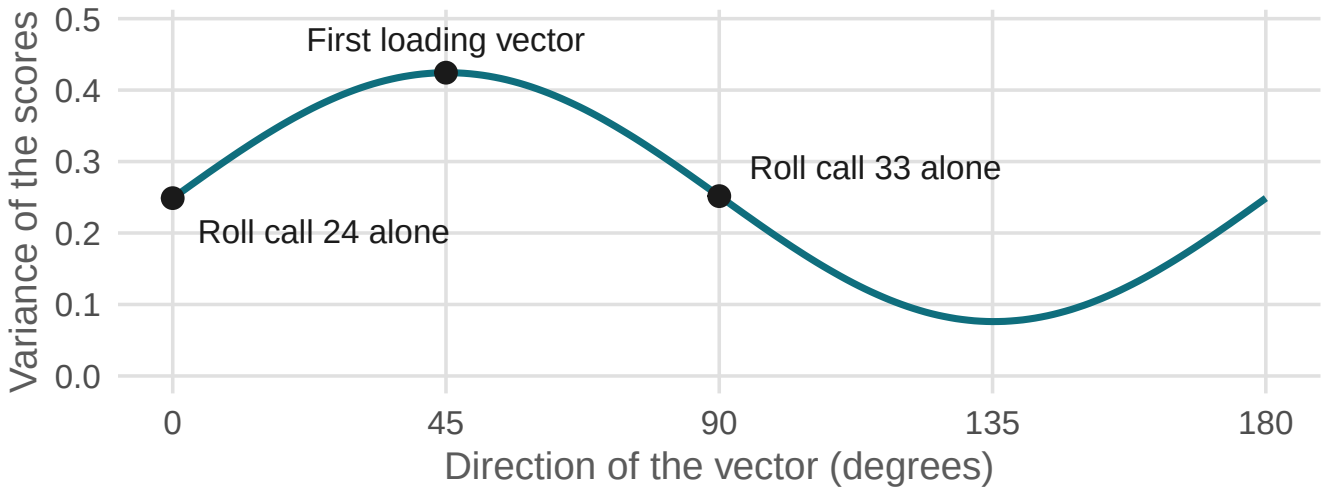
Many components

Selecting the loadings

Recall that we want to find the vector that separates legislators the most. That is, the vector whose scores have the highest spread. So we choose loadings that maximize the variance of the scores (subject to our constraint):

$$\max_{\phi_{11},\phi_{21}} \text{Var}(z_1) \quad \text{subject to} \quad \phi_{11}^2 + \phi_{21}^2 = 1$$

With two roll calls, we can simply try every direction:



The covariance matrix

With all $p = 300$ roll calls there are far too many directions to try, but there is an exact answer. It starts from the two roll calls' **covariance matrix**, Σ:

$$\Sigma = \begin{bmatrix} \text{Var}(y_1) & \text{Cov}(y_1, y_2) \\ \text{Cov}(y_2, y_1) & \text{Var}(y_2) \end{bmatrix} = \begin{bmatrix} 0.249 & 0.174 \\ 0.174 & 0.252 \end{bmatrix}$$

The **diagonal** holds each roll call's variance: how evenly it splits the legislature. For a yea/nay vote this is about $\bar{y}_j(1 - \bar{y}_j)$, at most 0.25, for a 50/50 split.

The **off-diagonal** holds their covariance: how much the two roll calls move together. It is positive because most legislators vote the same way on both.

Eigenvectors

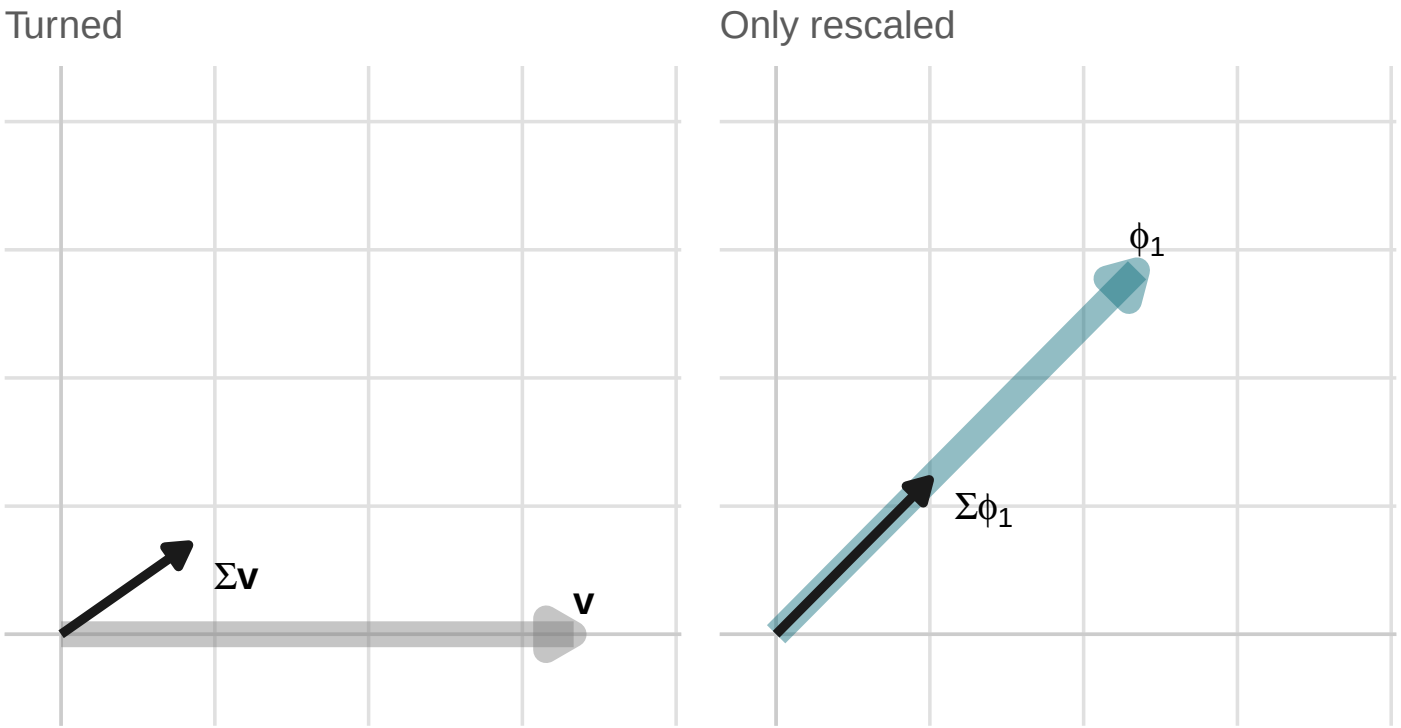
Applying Σ to a vector, $\Sigma\mathbf{v}$, usually turns it:

$$\Sigma \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.249 & 0.174 \\ 0.174 & 0.252 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.249 \\ 0.174 \end{bmatrix}$$

But some vectors it only rescales, without turning them. These are its **eigenvectors**, and the scale factor, λ, is the **eigenvalue**: $\Sigma\phi = \lambda\phi$.

$$\begin{bmatrix} 0.249 & 0.174 \\ 0.174 & 0.252 \end{bmatrix} \begin{bmatrix} 0.704 \\ 0.710 \end{bmatrix} = \begin{bmatrix} 0.299 \\ 0.301 \end{bmatrix} \approx 0.424 \begin{bmatrix} 0.704 \\ 0.710 \end{bmatrix}$$

Turned or only rescaled



ϕ_1 comes back pointing the same way, only shorter: $\Sigma\phi_1 = 0.424\,\phi_1$. That is what makes it an eigenvector.

Eigenvectors and PCA

This is how principal components analysis (PCA) finds its loadings: ϕ_1 is the eigenvector of Σ with the largest eigenvalue.

That eigenvalue, $\lambda_1 = 0.424$, is the variance of the scores along ϕ_1. The two roll calls' variances add to 0.50, so the first component explains 0.424 / 0.50 = 85%.

Eigenvectors are very useful: whenever a big matrix describes how things relate, its top eigenvector picks out the single most important pattern in it.

Roadmap

Learning without an outcome

One component

Finding the loadings

Many components

The first principal component

Let's do this properly, using all $p = 300$ roll calls:

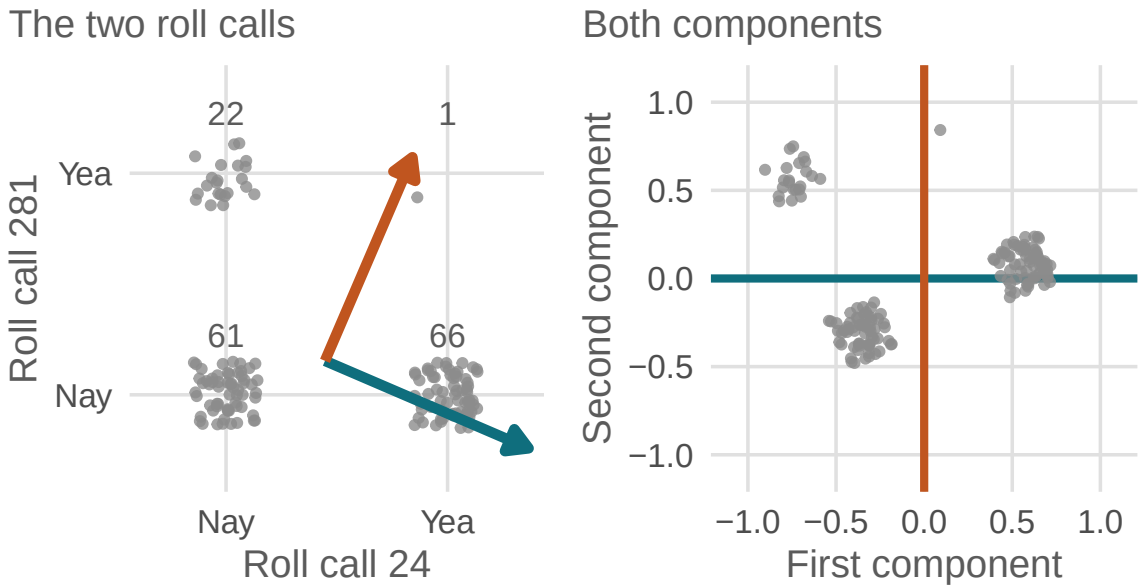
$$z_{i1} = \phi_{11}(y_{i1} - \bar{y}_1) + \phi_{21}(y_{i2} - \bar{y}_2) + \cdots + \phi_{p1}(y_{ip} - \bar{y}_p) = \sum_{j=1}^{p} \phi_{j1}(y_{ij} - \bar{y}_j)$$

The new variable z_1 is the **first principal component**; its values z_{i1} are the scores. Its loading vector ϕ_1, also called the principal component direction, is the first eigenvector of the 300×300 covariance matrix.

But is one dimension all there is?

A second dimension

Now pair roll call 24 with the less divisive roll call 281:



Who is in that corner? 19 of the 20 Left legislators from the North. On the first score they sit at the Left end, beyond the rest of the Left. Only the second score sets them apart from everyone.

The second principal component

The second score is another weighted sum, with its own loadings:

$$z_{i2} = \sum_{j=1}^{p} \phi_{j2} (y_{ij} - \bar{y}_j)$$

PCA chooses them the same way, maximizing the variance of the scores, with one more constraint:

$$\max_{\phi_{12},\ldots,\phi_{p2}} \text{Var}(z_2) \quad \text{subject to} \quad \sum_{j=1}^{p} \phi_{j2}^2 = 1 \quad \text{and} \quad \text{Cor}(z_1, z_2) = 0$$

Uncorrelated scores put the second loading vector at a right angle to the first, however many roll calls there are.

PCA in software

Different software finds the components in different ways, but gets the same answer:

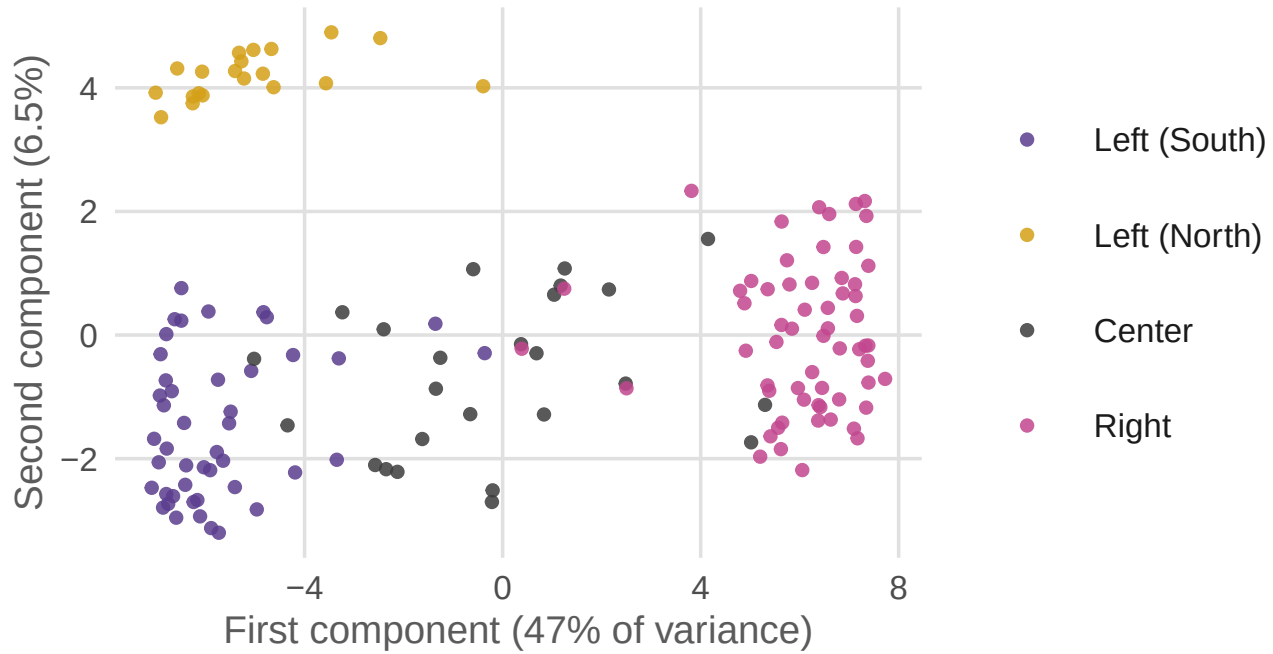
1. **Residualization**: regress every roll call's votes on the first scores, z_{i1}, and keep the residuals. The first component of those residuals is the second component. Repeat for the third, and so on (the NIPALS algorithm).
2. **Eigenvectors**: find every component at once, as the eigenvectors of Σ, ranked by their eigenvalues and all at right angles. R's `prcomp()` gets the same vectors from the votes directly.

The residuals in (1) are uncorrelated with z_{i1} by construction. That is how the constraint gets imposed.

But the signs can differ between packages: flipping every loading and score gives an equally good component. We fixed ours so that the Right scores high.

What cleavages exist?

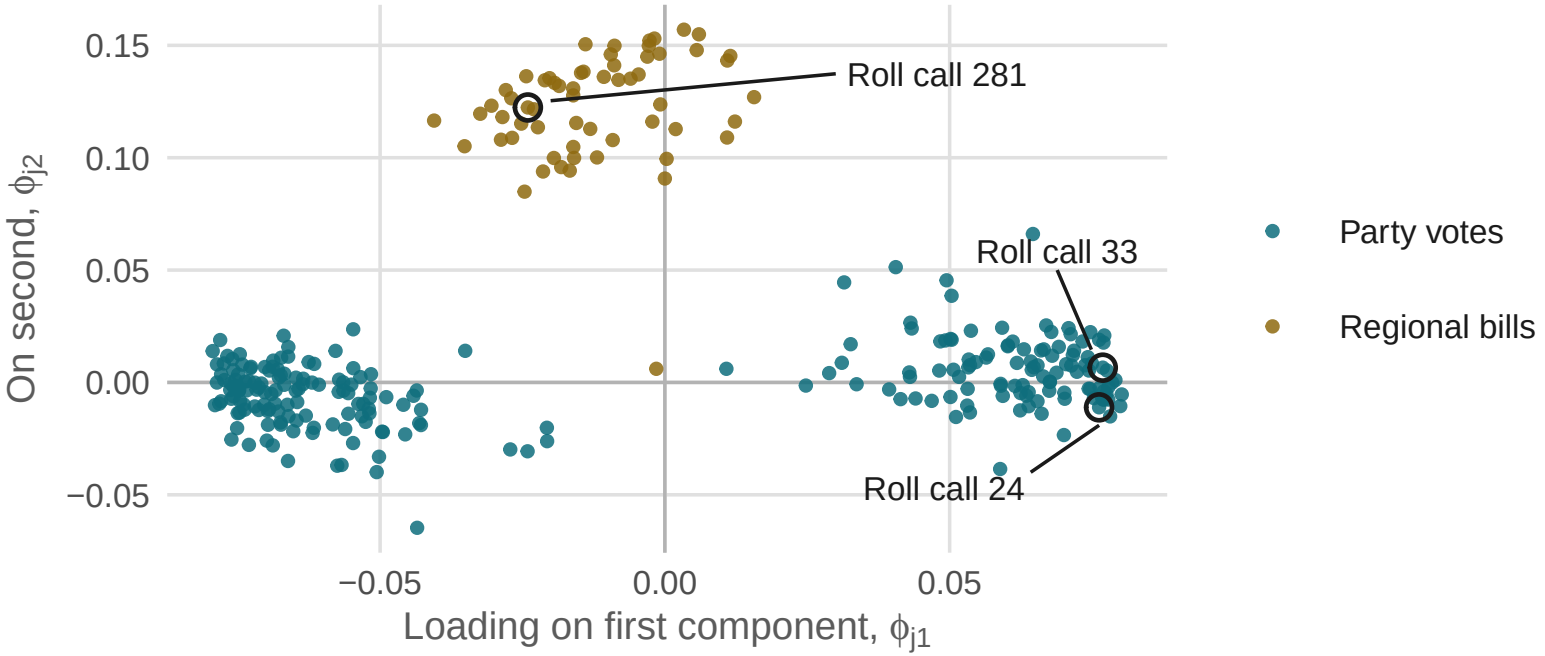
Now use all 300 roll calls. Each legislator becomes two scores instead of 300 votes:



The legislators fall along the first component, with one group set apart on the second. The first component orders the parties, Left to Right. The second picks out 20 Left legislators from the North.

Cleavages in the votes

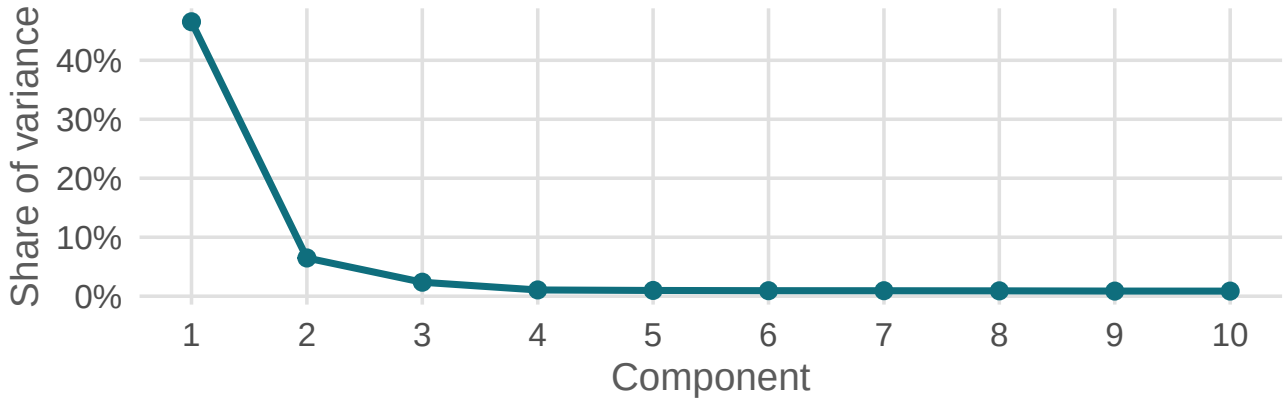
Each roll call has a loading on each component: its weight in that component's sum. Plot each roll call's two loadings:



We might label these dimensions 'left–right' and 'regional' because we know something about the politics of our country! PCA does not 'know' anything about substance.

How many components?

Each component explains a share of the total variance in the votes:



The first explains 47% and the second 6.5%. The third explains 2.4%, and the rest level off near 1%.

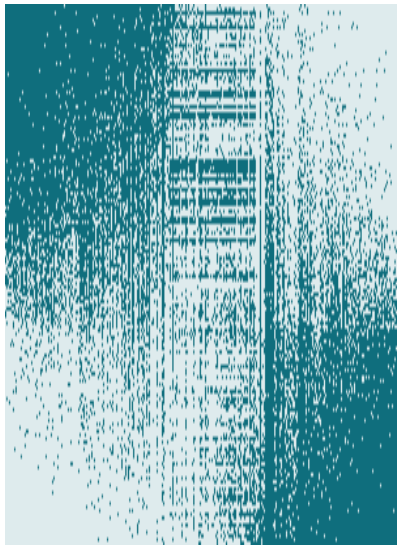
The curve has an **elbow**: after it, each extra component adds little. Here it comes at the second component, but where to cut is a judgment call.

We simulated two dimensions, so the third is an artifact.

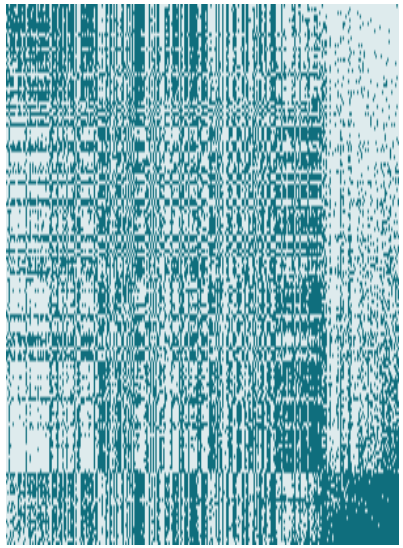
Sorting by component

Here are all 45,000 votes, in random order. Now sort the legislators by their scores, and the roll calls by their loadings, one component at a time:

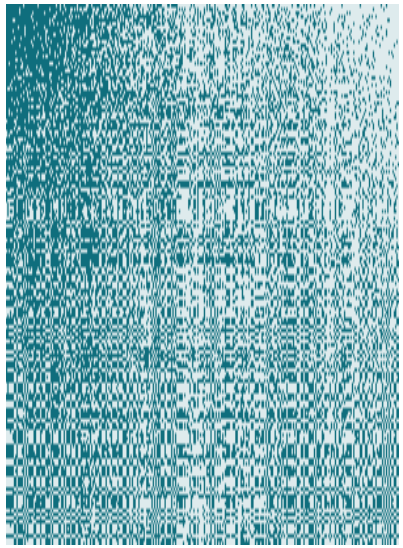
By the first



By the second



By the third



Side by side: each component sorts out less of the votes than the one before.

References

- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed. – **12.1–12.2.3**. statlearning.com.